1. MongoDB / JPA Data Model (Corrected)

Database design for HostelHub implementation

Critical Corrections & Missing Fields Added:
• **users Collection:** Added faceEmbedding (128D Float Vector Array/TEXT) & isFaceVerified (Boolean).
• **complaints Collection:** Added locationBlock, locationRoom, slaDeadline, slaBreached (Boolean), isMerged (Boolean), and parentTicketId.
• **work_logs / spare_parts:** Replaced raw string with structured parts table parts: [{partName, quantity, unitCost, lineTotal}] and totalCost.

users	complaints	work_logs / spare_parts
• _id (PK) • name • email • password • role [STUDENT|WARDEN|TECH] • **faceEmbedding (128D)** • **isFaceVerified** • studentId / roomNumber	• _id (e.g. #CH1025) • studentId • title & description • category & priority • status / stage • **locationBlock & room** • **slaBreached & isMerged** • rating & photos	• _id (PK) • complaintId (FK) • technicianId (FK) • partName & quantity • unitCost & lineTotal • **totalCost** • beforePhoto & afterPhoto

2. Java Class Hierarchy (Corrected)

Core Object-Oriented Structure & Polymorphism

Missing OOP Fields & Methods Added:

- **User <<abstract>>:** Added faceEmbedding: String and + verifyFace().
- **Student:** Added + reopenComplaint() and + submitComplaintWithPhoto().
- **Warden:** Attribute updated to blockAssigned. Added + mergeDuplicateComplaints() and + autoAssignTechnician().
- **Technician:** Attribute updated to specialty and activeTasksCount. Added + logSpareParts() and + markCompleted().

Student (extends User)	**Warden** (extends User)	**Technician** (extends User)
- studentId: String - hostelBlock: String - roomNumber: String + submitComplaint() + trackStatus() + rateService() **+ reopenComplaint()** **+ verifyFaceBiometrics()**	- blockAssigned: String - designation: String + viewComplaints() **+ autoAssignTechnician()** **+ mergeDuplicateComplaints()** + viewAnalytics() **+ escalateSLA()**	- specialty: String - activeTasksCount: int - avgRating: double + viewAssignedQueue() **+ logSpareParts()** **+ uploadAfterPhoto()** **+ markCompleted()**

3. State Pattern — Complaint Lifecycle (Corrected)

State Pattern controlling ticket lifecycle transitions

Missing Lifecycle Transitions Added:

1. **Forward Lifecycle:** SUBMITTED ■■■ UNDER REVIEW ■■■ ASSIGNED ■■■ IN PROGRESS ■■■ COMPLETED ■■■ CLOSED
2. **Reopen Loop:** From COMPLETED / CLOSED back to UNDER REVIEW / IN PROGRESS if student reopens an unresolved ticket.
3. **Duplicate Merge Shortcut:** Direct transition from SUBMITTED ■■■ CLOSED (MERGED) when warden merges duplicate complaints.

1. SUBMITTED Ticket created by student	**2. UNDER REVIEW** Warden triages ticket	**3. ASSIGNED** Tech assigned via Strategy
4. IN PROGRESS Tech starts work & parts log	**5. COMPLETED** After photo uploaded	**6. CLOSED** Rated or merged duplicate

4. Observer Pattern — Complaint Notifications

Dispatches real-time event updates to registered listeners

Missing Observer Added:
• The original slide omitted **WardenObserver**. Added WardenObserver to notify the Warden instantly when tickets are submitted, SLA deadlines breach, or duplicate complaints accumulate.

Subject: ComplaintSubject	Observers Registered
- status: Stage - observers: List<ComplaintObserver> + registerObserver() + removeObserver() + notifyStatusChange()	• **StudentNotifier:** Receives status transition alerts (e.g. Assigned ■■■ In Progress). • **TechnicianNotifier:** Receives new job queue assignments. • **WardenNotifier (NEW):** Receives SLA breach warnings & duplicate alerts.

5. Strategy Pattern — Technician Assignment

Smart technician workload balancing & specialty matching algorithms

Assignment Strategies Implemented:

- **LeastLoadedStrategy:** Inspects active tasks count of all technicians and assigns the candidate with minimum active workload.
- **SpecialtyStrategy:** Filters technicians by category specialty (e.g. Plumbing) first, then applies workload balancing.
- **AssignmentService:** Context class that dynamically executes the active strategy selected by the Warden.

Strategy 1: LeastLoadedStrategy	Strategy 2: SpecialtyStrategy
@Component("leastLoadedStrategy") Selects technician with minimum activeTasksCount to balance workload across staff.	@Component("specialtyStrategy") Matches complaint category (e.g. Electrical) to technician's specialty before fallback.

6. Factory Pattern — Complaint Creation (Recommended)

Centralized complaint object creation & auto-priority tagging

Factory Pattern Enhancement (ComplaintFactory):
• Instead of simple User creation, ComplaintFactory encapsulates business rules for ticket generation.
• **Auto-Priority Rule:** Category ELECTRICAL is automatically assigned **HIGH Priority** due to hazard risks.
• **Ticket ID Generation:** Generates unique CHxxxx identifier and sets initial state to SUBMITTED.

ComplaintFactory Input	ComplaintFactory Output
• Student Reference • Category (e.g. ELECTRICAL) • Description & Photos • Block & Room Location	• Unique ID: #CH1025 • Priority: **HIGH (Auto-tagged)** • Stage: **SUBMITTED** • Timestamp & SLA Deadline

7. Singleton Scope — Spring Beans

Spring Boot manages singleton scope by default for service components

Missing Singleton Services Added:

- **FaceVerificationService:** Singleton bean executing 128D face embedding cosine similarity calculations.
- **GeofenceService:** Singleton bean validating client IP against hostel Wi-Fi subnets.
- **ComplaintService:** Central singleton bean injected across controllers.

Spring Container (Singleton Scope Beans)
1. FaceVerificationService — Singleton for biometrics comparison 2. GeofenceService — Singleton for Wi-Fi IP subnet check 3. ComplaintService — Singleton for ticket business logic

8. Overall System Architecture

HostelHub — React + Spring Boot 3 + JPA / MySQL / H2

Complete System Stack:
React 18 Frontend ■■■ Spring Security (JWT + Face Auth) ■■■ Controllers ■■■ Services ■■■ Repositories ■■■ Database

1. React 18 Frontend: Role Switcher, Face Biometric Guard, Student/Warden/Tech views, Chart.js
2. Security & Biometrics Layer: Face-api.js + FaceVerificationService (128D) + Geofence IP check
3. Controllers Layer: AuthController, ComplaintController, WardenController, TechnicianController
4. Services & OOP Patterns: Factory, Strategy, Observer, State, Singleton services
5. Persistence & Cloud Layer: Spring Data JPA Repositories + MySQL/H2 DB + Cloudinary photo storage

9. Review Checklist & Viva Guide

Recommended presentation flow for project review

Step	Topic	Key Viva Defense Points
01	Database Data Model	Explain entity fields, 128D face vector storage, SLA breach flags, duplicate ticket references.
02	Java Class Hierarchy	Explain abstract User class, Student, Warden, Technician inheritance & encapsulation.
03	State Pattern	Demonstrate ticket lifecycle, Reopen loop from Completed, and Duplicate Merge shortcut.
04	Observer Pattern	Show real-time notifications dispatched to StudentObserver, TechnicianObserver & WardenObserver.
05	Strategy Pattern	Explain LeastLoadedStrategy vs SpecialtyStrategy for smart workload balancing.
06	Factory Pattern	Explain ComplaintFactory auto-assigning High Priority for Electrical hazard tickets.
07	Singleton Scope	Explain Spring-managed singletons: FaceVerificationService & GeofenceService.
08	System Architecture	Show full end-to-end integration: React ■■■ Spring Boot ■■■ JPA/MySQL ■■■ Cloudinary.